



A Case for *Managed* and *Model-less* Inference Serving

Neeraja J. Yadwadkar
Stanford University
neeraja@cs.stanford.edu

Qian Li
Stanford University
qianli@cs.stanford.edu

Francisco Romero
Stanford University
faromero@stanford.edu

Christos Kozyrakis
Stanford University, Google
kozyraki@stanford.edu

Abstract

The number of applications relying on inference from machine learning models, especially neural networks, is already large and expected to keep growing. For instance, Facebook applications issue tens-of-trillions of inference queries per day with varying performance, accuracy, and cost constraints. Unfortunately, today's inference serving systems are neither easy to use nor cost effective. Developers must manually match the performance, accuracy, and cost constraints of their applications to a large design space that includes decisions such as selecting the right model and model optimizations, selecting the right hardware architecture, selecting the right scale-out factor, and avoiding cold-start effects. These interacting decisions are difficult to make, especially when the application load varies over time, applications evolve over time, and the available resources vary over time.

If we want an increasing number of applications to use machine learning, we must automate issues that affect ease-of-use, performance, and cost efficiency for both users and providers. Hence, we define and make the case for *managed* and *model-less* inference serving. In this paper, we identify and discuss open research directions to realize this vision.

CCS Concepts • Computer systems organization → Cloud computing; • Computing methodologies → Machine learning.

Keywords Inference Serving, Model-less, Automatic Resource Management

ACM Reference Format:

Neeraja J. Yadwadkar, Francisco Romero, Qian Li, and Christos Kozyrakis. 2019. A Case for *Managed* and *Model-less* Inference

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HotOS '19, May 13–15, 2019, Bertinoro, Italy

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6727-1/19/05...\$15.00

<https://doi.org/10.1145/3317550.3321443>

Serving. In *Workshop on Hot Topics in Operating Systems (HotOS '19)*, May 13–15, 2019, Bertinoro, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3317550.3321443>

1 Introduction

Recent trends in analytics [11, 24, 29] indicate a considerable increase in the use of predictions from machine learning (ML) models, especially neural networks, by various applications. Examples include deep neural networks for video analytics and image recognition [29], natural language processing models [7, 47] for sentiment analysis tasks, and neural networks for scientific computing [30].

Typically, neural networks (models) have two distinct phases – training and inference. There has been a lot of work on accelerating convergence of the algorithms used for training, both from the ML [38, 45] and distributed systems [20, 50] communities. Various frameworks developed in industry, such as Google's TensorFlow [18] and Facebook's PyTorch [13], and in academia [36, 37, 46] allow for generation of accurate models using large training datasets. Training can take hours or days depending on multiple factors, including model architecture, and size of dataset used. Once models are trained, they can be queried by various end-user applications in the inference phase.

Inference serving is challenging, as it stresses both throughput and latency. For example, Facebook needs to serve tens-of-trillions of inference queries per day with real-time constraints [24]. This paper focuses on the four issues that make inference serving challenging in term of resource management. The challenges are summarized in Figure 1: (i) the diverse application requirements, (ii) the concurrent use of many model variants, (iii) the use of heterogeneous resources such as TPUs, GPUs, and CPUs, and (iv) the dynamic changes in application load and compute environments.

Diverse requirements: Applications have queries with different accuracy, cost, and latency requirements [24]. For instance, recommending and serving advertisements has stringent latency constraints, and can tolerate lower prediction accuracy, whereas speech recognition may be willing to trade-off latency for higher accuracy. Some applications produce millions of queries, while others generate only a few. Some queries are latency-sensitive (*Online*), while others are throughput-intensive (*Offline*). For example, real-time face

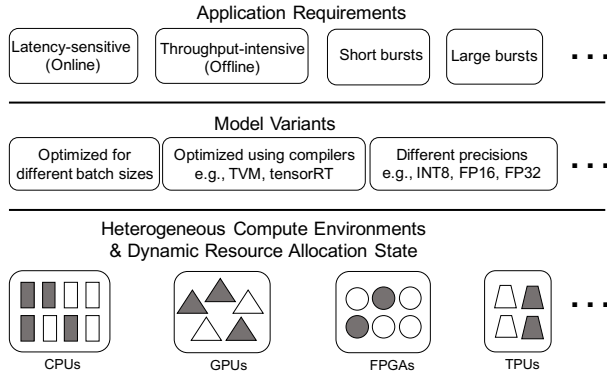


Figure 1. Variety in application requirements, model-variants, heterogeneous resources. Grayed-out boxes in the last layer show resources with models already loaded on them.

detection for security cameras is served online, whereas sentiment analysis on data collected over a week is processed offline. Moreover, application query patterns *change dynamically over time* (such as the diurnal patterns exhibited by face detection queries on Facebook), emphasizing the need for dynamically allocating resources to accommodate load unpredictability.

Model-variants: These diverse requirements lead to different models. Multiple *model-variants* can be generated by methods such as model compression [23, 26], knowledge distillation [33], or by changing the hyperparameter values. For instance, a teacher model [33] generated using a knowledge distillation method is more accurate than a student model, but requires larger memory and has a longer inference latency. Additionally, compiler optimizations, such as TVM [10] and TensorRT [41] produce variants of already trained models that are optimized for reduced inference latencies. Thus, variants of the same model may differ in their sizes, prediction accuracies, inference latencies, and resource-usage footprints [25]. We envision an inference serving system that selects a suitable model and its variant to satisfy the requirements of an application. Such an inference serving system will enable users to focus on querying an inference for their task without needing to think of the complex details of models and various model-variants.

Heterogeneous execution environments: The only way to meet diverse application requirements is to use the diverse set of hardware, such as CPUs, GPUs, TPUs, FPGAs, and other ASICs, that is available in datacenters today. These heterogeneous resources present different cost and performance profiles for model-variants [14, 27, 31]. For example, for a large batch of inference queries, GPUs typically show lower latencies than CPUs, but cost more and take longer to load models.

Dynamic changes in application load and compute environments: The choice of a model-variant and underlying compute environment is further complicated because – (i) Over time, application query rates can significantly vary (e.g., diurnal), and (ii) the compute environments change over time due to different coexisting applications [6]. Depending on the state of the models and the load from the applications, the resource management decisions will need to adjust.

If we expect applications to use ML inference, we need to make it simple-to-use and cost-effective for both users and providers. Despite significant research, this is missing right now. Some systems [11, 19, 40] require users to set up and manage their own resources, which is both expensive and burdensome. Inference services offered by cloud providers [3, 4, 16, 35] still require users to make critical decisions such as: the hardware resource(s) to use, the batch size, or even which variant of a model to query. To meet the stringent performance requirements of users, service providers often use dedicated resources to keep the models loaded and ready to be queried. This results in underutilized resources, limited hardware options, and scalability limitations – especially for bursty workloads.

In this paper, we articulate our vision for *managed* and *model-less* inference serving that leverages heterogeneous resources and multiple model-variants to meet diverse application requirements. We envision an inference serving system that automates resource provisioning for models to serve users’ inference queries in a way to satisfy their requirements. We call this a *managed* inference serving system. Going further, we argue for an interface to the managed inference serving system where users are able to focus on querying an inference for their tasks without needing to think of models, and the trade-offs offered by model-variants. We term this interface *model-less*. We note and draw attention to the potential of model-variants to satisfy diverse application requirements. We argue that such a system will make it simple to use for users, and reduce cost for both cloud providers and users. We identify and discuss open research directions to realize this vision.

2 Challenges

This section summarizes the gap between what existing inference serving solutions offer and the expectations from the user’s perspective.

2.1 Selecting a model-variant

Models differ in accuracy, inference latency, throughput, and resource-usage. The challenge of selecting a model is exacerbated by varying application requirements and no single model optimized for all cases [9]. Even after selecting the model, there is a large optimization space that leads to diverse model-variants. For instance, Figure 2 demonstrates

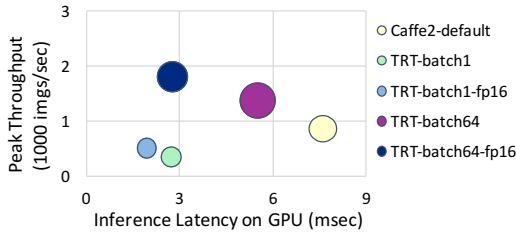


Figure 2. Inference times on batch size of 1 versus the maximum throughput (using the largest batch it supports, capped at batch size of 64) for ResNet50 variants (same accuracies) on NVIDIA V100. The circle area is proportional to the memory utilization. TRT - batchX denotes TensorRT optimization for batch size of X; fp16 denotes lower precision.

that given ResNet50, there is no best variant for all situations: the one optimized for batch-size of 1 suffers from low throughput, while the large batch optimization can lead to higher latency and memory utilization.

To select a model-variant, existing inference serving systems [11, 49] replicate a query and send it to multiple candidate models, leading to increased communication overhead and resource consumption.

Expectation 1: *Model-variant selection should be hidden behind a high level API that allows users to simply specify their performance and cost objectives.*

2.2 Heterogeneous hardware architectures

New types of ML hardware are frequently introduced, and the availability of hardware varies by vendor and time of the day. The choice of hardware, as well as the precision to use (e.g., FP32 vs Flexpoint [34] or FP16 vs bfloat16), has accuracy, cost, performance (latency or throughput), and energy implications. Compared to a CPU server, NVIDIA Tesla V100 GPU [39] provides 47× higher throughput, while the Xilinx U250 FPGA [51] improves the non-batch inference latency by 20×. A wide spectrum of hardware architectures, such as CPUs, GPUs, FPGAs and ASICs [27, 31], offer different performance and cost trade-offs for inference.

Figure 3 shows that GPUs usually have much lower latencies for large batch queries but with high loading overhead, while CPUs generally have lower load latencies and perform better with small batch sizes. However, there are exceptions: SqueezeNet1.1 performs better on CPU even for batch size of 64, and Transformer performs better on GPU for batch size of 1. As a result, the optimal batch size boundary between CPUs and GPUs is not fixed across models. Model load and inference latencies are dependent on the underlying hardware, and change based on the batch size being served. Furthermore, there are many compiler technologies [5, 10, 41, 43] that optimize inference latencies on different hardware accelerators without incurring loss in accuracy. Optimized models

produce additional model-variants with, for instance, different batch sizes or weight precisions. Thus, selecting the best hardware architecture, along with the optimal parameter values to meet an application’s cost constraints is tedious.

Most existing systems [4, 16] address this problem by fixing a target hardware and using a default model-variant while serving. Such systems ignore application requirements in terms of inference latencies. Other systems [11, 40] default to a model-variant that supports all batch sizes and use dynamic batching to balance throughput with latency constraints. However, using such default model has an impact on queries with stringent service level objective (SLO) requirements. We observed that a model-variant optimized for batch size of 64 can have 43% higher latency than the one optimized for a batch size of 1. We also note that for batch size of 64, a single precision variant on GPU can lead to 32% lower throughput compared to one using half-precision. Using a CPU results in up to 201× worse performance compared to the GPU counterpart. Misconfiguration or selecting a suboptimal model-variant can thus significantly impact the inference performance.

Expectation 2: *The choice of hardware should be hidden behind the same high level API for users. The system should select the right hardware type(s) to use at any point in time for meeting performance and cost SLOs.*

2.3 Varying load and query patterns

There are differences in load and query patterns, both across applications (popular vs. seldom-used, online vs. offline) and within applications (diurnal patterns). Latency-sensitive queries require a small number of resources for brief periods of time, while throughput-intensive jobs are longer-running and may scale across several different machines. Thus, to deal with the disparate resource management requirements of applications, cloud providers offer *separate* services for latency-sensitive and throughput-intensive [4, 16] that users need to configure. To minimize the complexity of user experience, we should not have users configure separate services for different applications or have users provision statically for their peak load. Moreover, to reduce the cost of using ML inference, the inference server should multiplex models as needed to better utilize resources.

The key to sharing and multiplexing models is minimizing the negative impact on performance [12, 28]. Figure 4 indicates that most models experience minimal performance loss when up to 5-6 concurrent instances are running, though the precise number of models that can share the GPU without interfering with each other is dependent on the batch sizes and peak memory footprints of the models. When collocating applications with different requirements and sharing model-variants, we must consider two query usage pattern characteristics – (i) at a given time, some models may be more popular than others, and (ii) models experience varying number of queries over time. In addition to challenges

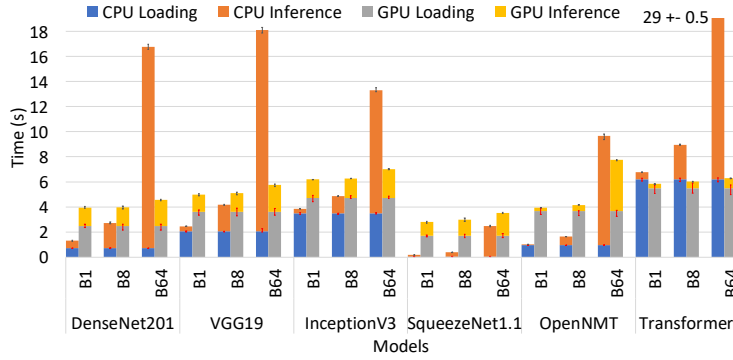


Figure 3. Loading and inference times of models built on PyTorch for batch sizes of 1, 8, and 64 on CPU and NVIDIA V100. The graph shows the average over 3 runs, and the whiskers show the minimum and maximum values observed. We note two trends: (i) CPUs generally load models faster than GPUs and (ii) CPUs have lower inference latencies for batch sizes less than 8.

mentioned in Sections 2.1 and 2.2, dynamically-changing query patterns emphasize the need for dynamic resource management mechanisms to address varying bursts of requests while maintaining the performance of already-issued queries.

To simplify resource provisioning, a common solution is to use both dedicated model instances per user and dedicated resources, such as GPUs. This leads to low resource utilization and increased total cost of ownership (TCO) for cloud-providers that is reflected on users. In a scenario where one model receives 1000 requests per second for one minute, we estimate that sharing across *both* CPUs and GPUs can result in **62×** cost savings compared to using AWS’s Elastic Inference `eia1.medium` instances [3], **6.2×** compared to using CPUs on persistent VMs, and **19.5×** cheaper compared to using AWS Lambda [8]. The cost savings are a result of (i) using CPUs until the model is loaded onto the GPUs (as oppose to exclusively using a CPU or a GPU), (ii) sharing up to 5 model replicas on one GPU (as oppose to using a dedicated GPU per model replica), and (iii) using the same model replica to service multiple requests (as oppose to spawning per-request replicas in the case of Amazon Lambda).

Expectation 3: *Resource management to meet query cost and performance goals for different models under varying load should be abstracted away from users. To improve provider resource utilization and TCO, the system must transparently (i) share resources across different model instances, (ii) share models across users, and (iii) manage memory by replicating and evicting models based on observed load and popularity.*

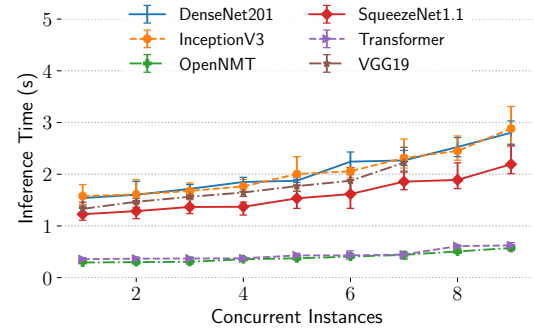


Figure 4. Inference times of model built on PyTorch for batch size of 1, spatially sharing one NVIDIA V100. The graph shows the average over 3 runs, and the whiskers show the minimum and maximum values observed. Most models experience minimal performance loss when up to 5-6 concurrent instances share the GPU.

2.4 Start-up latency

For inference serving, we define the *start-up latency* as the time for loading a model on a particular hardware platform. In contrast to VM or container cold-start [2, 42, 48], where the latency is more predictable for a specific instance, the start-up time for inference serving depends on the model-variant, the target hardware, the dynamic state of the system, and the framework being used (e.g., PyTorch, Caffe, or TensorFlow). The start-up latency not only manifests itself when scaling up (replicating model instances) [21] but also when adding a new model into the system. Hence, mitigating the start-up latency requires the system to take into account the current state of the system, as well as the time needed to load new models onto target hardware platforms.

A common approach to mitigating start-up latency is pre-loading and persisting models [11, 19, 40]. Suppose the VM price is C per hour and the system has M models. The utilized time percentage is P since the models are unlikely used 100% all the time. The estimated wasted resources per day would be $C \cdot M \cdot (1 - P) \cdot 24$. Now we assume $M = 100$, $P = 70\%$, and each model only has 1 replica. If all models are served on AWS Elastic Inference `eia1.medium`, the system would waste \$93.6 per day and **\$2,808** per month (30-days). If served on GCP preemptible 4-vCPU machines [17], the system wastes \$19.2 per day and **\$575** per month while likely having unacceptably high serving latencies. More importantly, as the number of models grows, the cost can easily double.

Expectation 4: *Start-up latency arising due to (i) loading a model-variant into the target hardware’s memory or storage and (ii) building an optimized model-variant, should be handled transparently.*

Expectation 5: *To prevent poor resource utilization, providers should not need to constantly keep model-variants running.*

3 Towards *managed* and *model-less* inference serving

The expectations from Section 2 suggest that inference serving should be simple-to-use for users and cost-efficient for both users and providers. To realize this vision, we believe inference serving needs to be *managed* and *model-less*. A user sends one or more queries for a prediction task, such as object recognition, with optional SLO constraints, for instance, 90th percentile inference latency. The serving system takes care of the rest – automatic model-variant and target hardware selection, automatic model-variant generation, load-aware adaptive scaling, fault tolerance, monitoring, logging, maintaining security and privacy of models and queries. This way, end-users or applications are neither required to think of selecting models or model-variants nor do they need to manage the resources for satisfying their performance and cost constraints. Going forward, we solicit research on several aspects of providing such a managed and model-less inference serving.

Automatic model-variant and hardware selection: As long as users have to reason about hardware differences and model characteristics, inference is not simple. Therefore, it is helpful for the system to automatically generate multiple variants of a submitted model and select the model-variant that satisfies application requirements. As explained earlier in the introduction, model-variants differ in their performance, cost, and resource usage footprints. Model-variants can be produced in various ways. Users can generate model-variants using techniques such as model compression [23, 26], or knowledge distillation [33]. Optimized versions of already trained models can be generated by compilers such as TVM [10] and TensorRT [41]. We have also observed that the same model trained on the same dataset using different frameworks, such as PyTorch, Caffe2, and TensorFlow, may result in variants that vary in inference latencies, and memory footprints.

The choice of model-variant impacts resource consumption and inference performance. Hence, we first need to understand characteristics of different model-variants. We then need to map the combination of model-variants and choice of target hardware to its impact on performance and cost. Based on a query's SLO, we need to map it to a 2-tuple – (model-variant, target hardware). To do so, we can auto-generate model-variants on demand, and profile previous runs of inference serving. Another open question for auto-generated model-variants is how to profile accuracy. This may require feedback on quality of served queries or a way for users to submit a representative set of test data to the system.

Dynamic placement of inference queries: Given SLO requirements, the mapping of a query to the aforementioned 2-tuple may need to change based on the dynamic state of the resources and models. The system can choose to generate a model-variant, load an existing one, or use an already-loaded instance. To make the final placement decisions that suit any query performance and cost objectives, we need to account for such dynamic state changes of model-variants and the underlying resource availability. For a latency-constrained query, this might entail selecting a model-variant that is already loaded. Conversely, for an offline batch of queries with throughput or cost constraints, we may prefer to select the optimal model-variant, even if it needs to be generated before being loaded.

Proactive startup optimization: Based on the placement decision, we may need to load a model, launch additional hardware resources, or evict loaded models to make resources available. This introduces start-up latency on the critical path of query placement and execution. How to design mechanisms and policies to avoid or reduce this latency remains an open question.

Inspired by existing resource management literature [15, 22], certain heuristic or learning-based optimizations should be performed proactively. Such optimizations include defining triggers based on observed utilization and expected load for scaling hardware resource and model instances up or down. We need replication policies to identify popular models based on query arrival patterns. We also need to design model-eviction policies to efficiently use available resources. How to predict load and changes in query submission pattern to design such policies remains to be explored.

Supporting fast and efficient decisions: Selecting suitable hardware for queries with SLOs requires navigating through the large space of static and dynamic options of model-variants and target hardware discussed earlier. This decision-making heavily depends on access to the dynamically changing state of the system to answer questions such as whether a model-variant is loaded on a particular hardware platform, how busy a machine is, or how many queries are being served by a model instance per time-unit. The question of how to capture and organize this data to allow fast access for decision-making needs to be explored.

Geo-distributed infrastructure for inference serving: With the emergence of edge and middle computing, in addition to the core (i.e., cloud datacenters), resource infrastructure is increasingly available at different vicinities from the application end-users [1, 44]. Different layers offer trade-offs in terms of resource capacity, cost and network latency, management overheads, and energy-efficiency. Utilizing the resources across this continuum of core-middle-edge computing opens new opportunities and research directions [32].

Model-variants can be loaded at different vantage points amongst these three tiers to support applications with diverse inference requirements. For instance, a model with low accuracy and memory footprint can be maintained at the edge to improve latency. If the application requires high accuracy predictions, we can direct the queries to a more accurate and resource-heavy model in the core at the cost of increased latency.

Managing model-variants placed in the cloud, in the middle, or at the edge thus present an opportunity to managed and model-less inference serving. Exploiting this opportunity poses various challenging questions: where should a model be loaded? How should query placement decisions be made? How and where should resource management decisions be made? Depending on an application's requirement, the system should trade off the proximity and reduced latency of the edge with the higher capacity of resources in the core.

Security and privacy: An inference serving system that is shared across multiple users may face privacy and data security concerns. We may need to develop mechanisms to prevent unauthorized access to queries, their submission patterns, and inference outcomes, to other users and the providers. Security mechanisms typically come at the cost of performance [52], i.e., we may be able to securely service a model, but it may incur a high latency. Thus, a managed and model-less inference serving system needs to select a model variant while accounting for the performance penalty of extracting inferences securely.

In many cases, personalized models are constructed by adding user-specific layers on top of a number of generic layers. To accelerate inference, computation of the generic layers can be shared across "similar" users, while the user-specific layers can be applied separately. However, depending on the privacy requirements of different users, such sharing of generic layers may not be feasible. How do we best support privacy for these kind of models? Do we treat them as separate models, or can we still take advantage of the generic layers for sharing? Answering these questions requires further assessment.

4 Conclusion

Inference serving faces new challenges given the need to process billions of ML-based application queries with diverse performance and cost requirements. The growing importance of ML inference forces us to finally solve several problems together: management of heterogeneity for both hardware and models, designing user interfaces, and building SLO-driven systems. These challenges are non-trivial, and create new avenues for research. The good news, however, is that it is a bounded problem (i.e., models and model-variants are immutable once created), thus giving us hope to get it right soon.

5 Acknowledgments

We thank Vivek Chawda, John Wilkes, Nipun Agarwala, Ana Klimovic, and other members of the MAST research group for helpful discussions regarding the project and for their insightful comments on drafts of this paper. This research is supported by Stanford Platform Lab and the industrial affiliates of Platform Lab.

References

- [1] M. AbdelBaky, M. Zou, A. R. Zamani, E. Renart, J. Diaz-Montes, and M. Parashar. 2017. Computing in the Continuum: Combining Pervasive Devices and Services to Support Data-Driven Applications. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. 1815–1824. <https://doi.org/10.1109/ICDCS.2017.323>
- [2] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. 2018. SAND: Towards High-Performance Serverless Computing. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 923–935. <https://www.usenix.org/conference/atc18/presentation/akkus>
- [3] Amazon 2018. Amazon Elastic Inference. <https://aws.amazon.com/machine-learning/elastic-inference/>.
- [4] Amazon 2018. Amazon SageMaker. <https://aws.amazon.com/sagemaker/>.
- [5] Amazon 2018. Amazon SageMaker Neo. <https://aws.amazon.com/sagemaker/neo/>.
- [6] Dan Ardelean, Amer Diwan, and Chandra Erdman. 2018. Performance Analysis of Cloud Applications. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 405–417. <https://www.usenix.org/conference/nsdi18/presentation/ardelean>
- [7] Mohammed Attia, Younes Samih, Ali Elkahky, and Laura Kallmeyer. 2018. Multilingual Multi-class Sentiment Classification Using Convolutional Neural Networks. Miyazaki, Japan, 635–640. <http://www.lrec-conf.org/proceedings/lrec2018/pdf/149.pdf>
- [8] AWS 2018. AWS Lambda. <https://aws.amazon.com/lambda/>.
- [9] Alfredo Canziani, Adam Paszke, and Eugenio Culurciello. 2016. An Analysis of Deep Neural Network Models for Practical Applications. *CoRR* abs/1605.07678 (2016). arXiv:1605.07678 <http://arxiv.org/abs/1605.07678>
- [10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. <https://www.usenix.org/conference/osdi18/presentation/chen>
- [11] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2017, Boston, MA, USA, March 27-29, 2017*. 613–627. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>
- [12] Abdul Dakkak, Cheng Li, Simon Garcia De Gonzalo, Jinjun Xiong, and Wen-Mei W. Hwu. 2018. TrIMS: Transparent and Isolated Model Sharing for Low Latency Deep Learning Inference in Function as a Service Environments. *CoRR* abs/1811.09732 (2018). arXiv:1811.09732 <http://arxiv.org/abs/1811.09732>
- [13] Facebook 2017. PyTorch. <https://pytorch.org/>.
- [14] Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Todd Masegill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, Logan Adams, Mahdi Ghandi, Stephen Heil, Prerak Patel, Adam

- Sapek, Gabriel Weisz, Lisa Woods, Sitaram Lanka, Steven K. Reinhardt, Adrian M. Caulfield, Eric S. Chung, and Doug Burger. 2018. A Configurable Cloud-scale DNN Processor for Real-time AI. In *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA '18)*. IEEE Press, Piscataway, NJ, USA, 1–14. <https://doi.org/10.1109/ISCA.2018.00012>
- [15] Anshul Gandhi, Mor Harchol-Balter, Ram Raghunathan, and Michael A Kozuch. 2012. Autoscale: Dynamic, robust capacity management for multi-tier data centers. *ACM Transactions on Computer Systems (TOCS)* 30, 4 (2012), 14.
- [16] Google 2018. Google Cloud Machine Learning Engine. <https://cloud.google.com/ml-engine/>.
- [17] Google 2018. Google Compute Engine Pricing. <https://cloud.google.com/compute/pricing>.
- [18] Google 2018. *TensorFlow - An open source machine learning framework for everyone*. <https://www.tensorflow.org>.
- [19] Google 2018. *TensorFlow Serving for model deployment in production*. <https://www.tensorflow.org/serving/>.
- [20] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU Cluster Manager for Distributed Deep Learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, 485–500. <https://www.usenix.org/conference/nsdi19/presentation/gu>
- [21] Arpan Gujarati, Sameh Elnikety, Yuxiong He, Kathryn S McKinley, and Björn B Brandenburg. 2017. Swayam: distributed autoscaling to meet SLAs of machine learning inference services with resource efficiency. In *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference*. ACM, 109–120.
- [22] Arpan Gujarati, Sameh Elnikety, Yuxiong He, Kathryn S. McKinley, and Björn B. Brandenburg. 2017. Swayam: distributed autoscaling to meet SLAs of machine learning inference services with resource efficiency. In *Middleware*. ACM, 109–120.
- [23] Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. In *Advances in neural information processing systems*. 1135–1143.
- [24] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, James Law, Kevin Lee, Jason Lu, Pieter Noordhuis, Misha Smelyanskiy, Liang Xiong, and Xiaodong Wang. 2018. Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective. In *Proceedings of the 2018 IEEE International Symposium on High Performance Computer Architecture (HPCA) (HPCA '18)*. IEEE. <https://doi.org/10.1109/HPCA.2018.00059>
- [25] Gao Huang, Danlu Chen, Tianhong Li, Felix Wu, Laurens van der Maaten, and Kilian Weinberger. 2018. Multi-Scale Dense Networks for Resource Efficient Image Classification. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=Hk2almxAb>
- [26] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. 2016. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 mb model size. *arXiv preprint arXiv:1602.07360* (2016).
- [27] Intel 2018. Intel Nervana Neural Network Processor. <https://ai.intel.com/nervana-nnp/>.
- [28] Paras Jain, Xiangxi Mo, Ajay Jain, Harikaran Subbaraj, Rehan Durrani, Alexey Tumanov, Joseph Gonzalez, and Ion Stoica. 2018. Dynamic Space-Time Scheduling for GPU Inference. In *LearningSys Workshop at Neural Information Processing Systems 2018*.
- [29] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. 2018. Chameleon: Scalable Adaptation of Video Analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. ACM, New York, NY, USA, 253–266. <https://doi.org/10.1145/3230543.3230574>
- [30] Eric Jonas, Qifan Pu, Shivaram Venkataraman, Ion Stoica, and Benjamin Recht. 2017. Occupy the Cloud: Distributed Computing for the 99%. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*. ACM, New York, NY, USA, 445–451. <https://doi.org/10.1145/3127479.3128601>
- [31] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datcenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. ACM, New York, NY, USA, 1–12. <https://doi.org/10.1145/3079856.3080246>
- [32] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (2017-01-01) (ASPLOS '17)*. ACM, Xi'an, China, 615–629. <https://doi.org/10.1145/3037697.3037698>
- [33] Animesh Koratana, Daniel Kang, Peter Bailis, and Matei Zaharia. 2018. LIT: Block-wise Intermediate Representation Training for Model Compression. *CoRR abs/1810.01937* (2018). arXiv:1810.01937 <http://arxiv.org/abs/1810.01937>
- [34] Urs Köster, Tristan Webb, Xin Wang, Marcel Nassar, Arjun K. Bansal, William Constable, Oguz Elibol, Stewart Hall, Luke Hornof, Amir Khosrowshahi, Carey Kloss, Ruby J. Pai, and Naveen Rao. 2017. Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks. *CoRR abs/1711.02213* (2017). arXiv:1711.02213 <http://arxiv.org/abs/1711.02213>
- [35] Microsoft 2018. *Azure Machine Learning*. <https://docs.microsoft.com/en-us/azure/machine-learning/>.
- [36] Philipp Moritz, Robert Nishihara, Ion Stoica, and Michael I. Jordan. 2015. SparkNet: Training Deep Networks in Spark. *CoRR abs/1511.06051* (2015). arXiv:1511.06051 <http://arxiv.org/abs/1511.06051>
- [37] MXNet 2017. *Apache MXNet (Incubating) - A flexible and efficient library for deep learning*. <https://mxnet.apache.org/>.
- [38] Feng Niu, Benjamin Recht, Christopher Re, and Stephen J. Wright. 2011. HOGWILD!: A Lock-free Approach to Parallelizing Stochastic Gradient Descent. In *Proceedings of the 24th International Conference on Neural Information Processing Systems (NIPS'11)*. Curran Associates Inc., USA, 693–701. <http://dl.acm.org/citation.cfm?id=2986459.2986537>
- [39] NVIDIA 2017. *NVIDIA Tesla V100 Tensor Core GPU*. <https://www.nvidia.com/en-us/data-center/tesla-v100/>.
- [40] NVIDIA 2018. *NVIDIA TensorRT Inference Server*. <https://github.com/NVIDIA/tensorrt-inference-server>.
- [41] NVIDIA 2018. *NVIDIA TensorRT: Programmable Inference Accelerator*. <https://developer.nvidia.com/tensorrt>.
- [42] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK:

- Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>
- [43] Young H. Oh, Quan Quan, Daeyeon Kim, Seonghak Kim, Jun Heo, Sungjun Jung, Jaeyoung Jang, and Jae W. Lee. 2018. A Portable, Automatic Data Quantizer for Deep Neural Networks. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (PACT '18)*. ACM, New York, NY, USA, Article 17, 14 pages. <https://doi.org/10.1145/3243176.3243180>
- [44] Mahadev Satyanarayanan. 2017. The Emergence of Edge Computing. *Computer* 50, 1 (Jan. 2017), 30–39. <https://doi.org/10.1109/MC.2017.9>
- [45] Virginia Smith, Simone Forte, Chenxin Ma, Martin Takác, Michael I. Jordan, and Martin Jaggi. 2017. CoCoA: A General Framework for Communication-Efficient Distributed Optimization. *Journal of Machine Learning Research* 18 (2017), 230:1–230:49. [http://jmlr.org/papers/v18/papers/v18/16-512.html](http://jmlr.org/papers/v18/papers/v18/papers/v18/16-512.html)
- [46] Leonard Truong, Rajkishore Barik, Ehsan Totoni, Hai Liu, Chick Markley, Armando Fox, and Tatiana Shpeisman. 2016. Latte: A Language, Compiler, and Runtime for Elegant and Efficient Deep Neural Networks. *SIGPLAN Not.* 51, 6 (June 2016), 209–223. <https://doi.org/10.1145/2980983.2908105>
- [47] Leonid Velikovich, Ian Williams, Justin Scheiner, Petar S. Aleksic, Pedro J. Moreno, and Michael Riley. 2018. Semantic Lattice Processing in Contextual Automatic Speech Recognition for Google Assistant. In *Interspeech 2018, 19th Annual Conference of the International Speech Communication Association, Hyderabad, India, 2-6 September 2018*. 2222–2226. <https://doi.org/10.21437/Interspeech.2018-2453>
- [48] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. 2018. Peeking Behind the Curtains of Serverless Platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, Boston, MA, 133–146. <https://www.usenix.org/conference/atc18/presentation/wang-liang>
- [49] Wei Wang, Jinyang Gao, Meihui Zhang, Sheng Wang, Gang Chen, Teck Khim Ng, Beng Chin Ooi, Jie Shao, and Moaz Reyad. 2018. Rafiki: machine learning as an analytics service system. *Proceedings of the VLDB Endowment* 12, 2 (2018), 128–140.
- [50] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. 2018. Gandiva: Introspective Cluster Scheduling for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 595–610. <https://www.usenix.org/conference/osdi18/presentation/xiao>
- [51] Xilinx. 2018. *Accelerating DNNs with Xilinx Alveo Accelerator Cards*. https://www.xilinx.com/support/documentation/white_papers/wp504-accel-dnns.pdf.
- [52] Mengjia Yan, Christopher W. Fletcher, and Josep Torrellas. 2018. Cache Telepathy: Leveraging Shared Resource Attacks to Learn DNN Architectures. *CoRR* abs/1808.04761 (2018). arXiv:1808.04761 <http://arxiv.org/abs/1808.04761>